

Directing Measures and Markov Mixtures:

Formalizing Diaconis–Freedman 1980 in Lean 4

Mettapedia Project

April 2026

Abstract

We present a complete Lean 4 formalization of the Markov de Finetti theorem (Diaconis–Freedman 1980): every Markov-exchangeable, recurrent prefix measure on a finite alphabet is a mixture of time-homogeneous Markov chains. The formalization comprises 26,665 lines of Lean code with zero unproved assumptions (`sorry`), building on a 43,683-line formalization of the classical (i.i.d.) de Finetti theorem. Key innovations include a directing measure uniqueness lemma that transfers de Finetti structure across measure restrictions, and a carrier transport bijection that establishes start-state invariance via trajectory segment swapping. The proof architecture decomposes into successor-matrix partial exchangeability, per-row de Finetti factorization, and cross-row coherence induction. This formalization provides a verified foundation for structured universal mixtures in the style of Solomonoff induction, where Markov models replace i.i.d. hypotheses for tractable sequential prediction.

Contents

1	Introduction	3
1.1	The Theorem	3
1.2	Why Formalize?	3
1.3	Contribution	3
2	Mathematical Background	4
2.1	Prefix Measures	4
2.2	Transition Counts and Markov Evidence	4
2.3	Markov Exchangeability	4
2.4	The Markov Parameter Space	5
2.5	Recurrence	5
3	The Formal Statement	5
3.1	Row-Kernel Crux Data	5
3.2	The Internal Representation Theorem	6
3.3	The Crown Theorem	6
4	Proof Architecture	6
4.1	Layer 1: Successor-Matrix Partial Exchangeability	7
4.2	Layer 2: Row-Kernel Construction	7
4.3	The Directing Measure Uniqueness Lemma	7
4.4	Start-Restricted Factorization	7
4.5	Carrier Transport	8

5	Key Lemmas	8
5.1	Directing Measure Uniqueness	8
5.2	Start-Restricted Row-Kernel Data	9
5.3	Carrier Transport Bijection	9
6	File Inventory	9
6.1	External Dependencies	9
7	The Classical de Finetti Context	10
7.1	Row-Process Decomposition	10
8	Toward Solomonoff: Structured Universal Mixtures	11
8.1	The Solomonoff Connection	11
8.2	Dominance Guarantees	11
8.3	PLN Evidence Aggregation	11
8.3.1	The PLN-Markov Bridge	11
8.3.2	The Markov Xi Transition Surface	12
9	The Structural Audit: Bug-Hunting Methodology	12
9.1	The Problem: Assumption Interfaces Mask Gaps	13
9.2	Bug 1: Evidence-Preserving Equiv Used Below Full Strength	13
9.3	Bug 2: Zero-Sorry Masking of Unproved Content	13
9.4	Bug 3: The Euler Trail Archive Was the Missing Bridge	13
9.5	Audit Methodology	14
10	Completed Extensions	14
10.1	Abstract Recurrence Generalization	14
10.2	The MarkovMixture API	15
10.3	Kleisli(Giry) Prefix-Law Bridge	15
10.4	Binary Post-Markov Bridge	15
10.5	Borel-Measurable Sequence Kernel	16
10.6	Categorical Borel-Side Surface	16
10.7	Non-Uniqueness of Borel Lifts	17
10.8	Higher-Order Markov Chains	17
10.9	Finite Hidden Markov Models	19
10.10	Higher-Order Borel/Giry Bridge	20
10.11	Finite HMM de Finetti Bridge	20
10.12	Controlled Finite HMMs, Credal Beliefs, and POMDP-Style Planning	20
10.13	Controlled Solomonoff Regret Along Fixed Action Streams	21
10.14	Higher-Order WM/PLN Bridge	22
11	Conclusion	22
11.1	Lessons Learned	22
11.2	Future Directions	23
11.2.1	Existence of Non-Unique Borel Lift	23
11.2.2	Computational Extraction	23
11.2.3	Extended State Spaces	23

1 Introduction

Solomonoff induction (1964) provides a universal Bayesian mixture over all computable hypotheses. In practice, one restricts to structured hypothesis classes—i.i.d. models, finite-state Markov models, hidden Markov models—for tractability and interpretability. The conceptual bridge is:

Exchangeability (or partial exchangeability) assumptions identify when a rich family of distributions can be represented as a mixture over a simpler parametric family.

For i.i.d. sequences, de Finetti’s theorem (1931/1937) gives exchangeability $\Rightarrow$ mixture of i.i.d. laws. For Markov sequences, Diaconis and Freedman (1980) show a corresponding representation for *Markov-exchangeability*, with an additional *recurrence* hypothesis.

1.1 The Theorem

Let $A = \{0, 1, \dots, k-1\}$ be a finite alphabet. A *prefix measure* μ assigns a nonnegative extended real to each finite word over A , satisfying $\mu(\varepsilon) = 1$ and $\mu(xs) = \sum_{a \in A} \mu(xs \cdot [a])$. We say μ is *Markov-exchangeable* if $\mu(xs) = \mu(ys)$ whenever xs and ys have the same *Markov evidence*: the starting state and transition-count matrix.

Theorem 1.1 (Diaconis–Freedman 1980). Let μ be a Markov-exchangeable prefix measure on a finite alphabet A . If μ satisfies a recurrence condition (the trajectory returns to its starting state infinitely often, almost surely under any extension), then there exists a probability measure π on the space of Markov parameters $\Theta_k = \Delta_k \times \Delta_k^k$ such that for all finite words xs :

$$\mu(xs) = \int_{\Theta_k} \text{wordProb}_{\theta}(xs) d\pi(\theta)$$

where $\text{wordProb}_{\theta}(x_0, x_1, \dots, x_n) = \pi_0(x_0) \cdot \prod_{t=0}^{n-1} P(x_t, x_{t+1})$ is the Markov chain probability.

1.2 Why Formalize?

The Markov de Finetti theorem is foundational for:

- **Bayesian sequential prediction:** Dirichlet-multinomial conjugacy for Markov chains, with dominance-style guarantees.
- **PLN evidence aggregation:** The “evidence is what you carry” design—under exchangeability, binary counts (n^+, n^-) are sufficient statistics.
- **Structured universal mixtures:** Solomonoff-like prediction instantiated over finite-state hypotheses with hyperpriors retaining universality guarantees.

The proof involves subtle measure-theoretic arguments: conditional independence across row processes, almost-everywhere uniqueness of directing measures, and carrier bijections preserving transition counts. Formalization provides certainty that the intricate decomposition is correct.

1.3 Contribution

We present:

1. A complete formalization of Theorem 1.1 in Lean 4 with Mathlib: **26,665 lines, 0 sorry, 33 files**.
2. Reuse of a 43,683-line formalization of the classical de Finetti theorem (three independent proofs: martingale, L^2 , ergodic).

3. Key innovations:

- *Directing measure uniqueness*: If $\nu \leq C \cdot \mu$ for exchangeable measures, their directing measures agree ν -a.e.
- *Carrier transport*: Trajectory segment swapping provides evidence-preserving bijections between visit-time fibers.

4. A structural audit methodology that identified and closed three previously overlooked gaps in the proof architecture.

2 Mathematical Background

2.1 Prefix Measures

A *prefix measure* on $\text{Fin}(k)$ is a function $\mu : \text{List}(\text{Fin}(k)) \rightarrow [0, \infty]$ satisfying:

- $\mu([]) = 1$ (normalization)
- $\mu(xs) = \sum_{a \in \text{Fin}(k)} \mu(xs \cdot [a])$ (consistency)

In Lean:

```
-- FiniteAlphabet.PrefixMeasure (Fin k)
-- A function List (Fin k) -> ENNReal satisfying consistency
```

2.2 Transition Counts and Markov Evidence

For a trajectory $xs : \text{Fin}(n+1) \rightarrow \text{Fin}(k)$, the *Markov evidence* consists of:

- The starting state x_0
- The transition-count matrix $N(i, j) = \#\{t < n : x_t = i \wedge x_{t+1} = j\}$

```
structure MarkovEvidence (k : Nat) where
  start : Fin k
  trans : Fin k -> Fin k -> Nat

def evidenceOf (xs : Fin (n + 1) -> Fin k) : MarkovEvidence k :=
{ start := xs 0,
  trans := fun a b => transCount xs a b }
```

2.3 Markov Exchangeability

Definition 2.1. A prefix measure μ is *Markov-exchangeable* if for all xs, ys of the same length with $\text{evidenceOf}(xs) = \text{evidenceOf}(ys)$: $\mu(xs) = \mu(ys)$.

```
def MarkovExchangeablePrefixMeasure
  (mu : PrefixMeasure (Fin k)) : Prop :=
forall (n : Nat) (xs1 xs2 : Fin (n + 1) -> Fin k),
  evidenceOf xs1 = evidenceOf xs2 ->
  mu (List.ofFn xs1) = mu (List.ofFn xs2)
```

2.4 The Markov Parameter Space

A Markov model on $\text{Fin}(k)$ is parameterized by:

- An initial distribution $\pi_0 \in \Delta_k$
- k row-stochastic transition distributions $P(i, \cdot) \in \Delta_k$

The parameter space $\Theta_k = \Delta_k \times \Delta_k^k$ is a finite product of simplices, hence compact. The word probability function $\theta \mapsto \text{wordProb}_\theta(xs)$ is continuous on Θ_k .

2.5 Recurrence

Markov exchangeability alone does *not* imply a mixture representation. A counterexample: the deterministic chain $0 \rightarrow 1 \rightarrow 1 \rightarrow 1 \rightarrow \dots$ is Markov-exchangeable (trivially—equivalence classes are singletons) but not a mixture of recurrent Markov chains.

Definition 2.2. A prefix measure μ is *recurrent* if it admits an extension P to infinite sequences where the trajectory returns to its starting state infinitely often, P -almost surely.

```
def recurrentEvent : Set (Nat -> Fin k) :=
  { omega | forall N, exists n >= N, omega n = omega 0 }

def MarkovRecurrentPrefixMeasure (mu : PrefixMeasure (Fin k)) : Prop :=
  exists (P : Measure (Nat -> Fin k)),
    IsProbabilityMeasure P /\
    (forall xs, mu xs = P (cylinder xs)) /\
    P recurrentEvent = 1
```

3 The Formal Statement

3.1 Row-Kernel Crux Data

The internal representation theorem requires *row-kernel crux data*: a family of row kernels with measurability, start-restriction, and coherence properties.

```
def RowKernelCruxData
  (P : Measure (Nat -> Fin k))
  (rowKernel : Fin k -> (Nat -> Fin k)
    -> ProbabilityMeasure (Fin k)) : Prop :=
  -- (1) Singleton-eval AE-measurability
  (forall i b, AEMeasurable
    (fun r => (rowKernel i r) {b})
    (rowProcessLaw P i)) /\
  -- (2) Start-restricted invariance
  StartRestrictedRowKernelData P rowKernel /\
  -- (3) Product-kernel AE-measurability
  (forall i, AEMeasurable
    (fun r => Measure.pi (fun _ => rowKernel i r))
    (rowProcessLaw P i)) /\
  -- (4) Cross-row coherence step
  CrossRowCoherenceStep P rowKernel
```

3.2 The Internal Representation Theorem

Theorem 3.1 (Internal Representation, proved). Given an extension P of a prefix measure μ with row-kernel crux data, there exists a probability measure π on $\text{MarkovParam}(k)$ such that $\mu(xs) = \int \text{wordProb}_\theta(xs) d\pi(\theta)$.

```

theorem fortiniSuccessorMatrixInvarianceTheoremInternal_proved :
  FortiniSuccessorMatrixInvarianceTheoremInternal k := by
  intro mu hP hExt rowKernel hcrux
  -- Construct pi as pushforward of P through
  -- omega /-> rowKernelToMarkovParam(omega)
  ...

```

3.3 The Crown Theorem

The full theorem derives the row-kernel crux data from Markov exchangeability and recurrence alone.

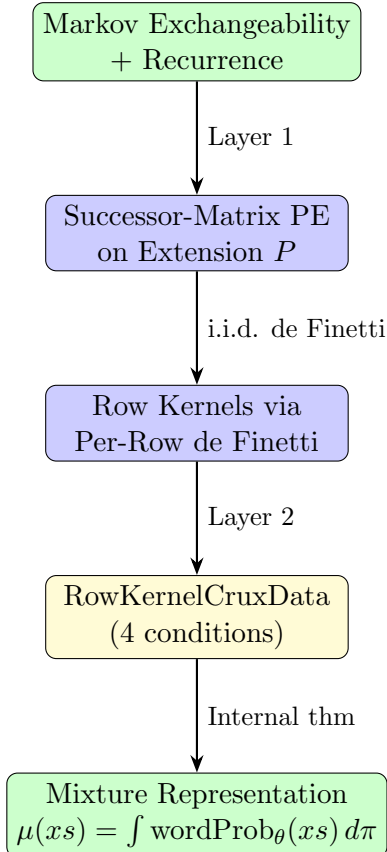
```

abbrev FortiniCanonicalRepresentationTheorem (k : Nat) : Prop :=
  forall mu : PrefixMeasure (Fin k),
    MarkovExchangeablePrefixMeasure mu ->
    MarkovRecurrentPrefixMeasure mu ->
      exists (pi : Measure (MarkovParam k)),
        IsProbabilityMeasure pi /\
        forall xs, mu xs = integral theta, wordProb theta xs dpi

```

4 Proof Architecture

The proof decomposes into two layers:



4.1 Layer 1: Successor-Matrix Partial Exchangeability

A measure P on $\mathbb{N} \rightarrow \text{Fin}(k)$ is *successor-matrix partially exchangeable* if for every finite selection of anchor states and visit indices, within-row permutations leave the joint law invariant.

```
def SuccessorMatrixPartialExchangeable
  (P : Measure (Nat -> Fin k)) : Prop :=
  forall (m : Nat) (anchor : Fin m -> Fin k)
    (idx : Fin m -> Nat) (sigma : Fin k -> Equiv.Perm Nat),
    Measure.map
      (fun omega => fun j => rowSuccessorVisitProcess (anchor j) omega
        ((sigma (anchor j)) (idx j))) P
  =
  Measure.map
    (fun omega => fun j => rowSuccessorVisitProcess (anchor j) omega
      (idx j)) P
```

This is derived from Markov exchangeability via the carrier transport argument.

4.2 Layer 2: Row-Kernel Construction

From successor-matrix PE, each row's law $\text{rowProcessLaw}(P, i)$ is exchangeable in the classical sense. We apply the i.i.d. de Finetti theorem to extract the *directing measure*:

```
noncomputable def directingRowKernel
  (P : Measure (Nat -> Fin k)) (i : Fin k)
  (r : Nat -> Fin k) : ProbabilityMeasure (Fin k) :=
  directingMeasure (mu := rowProcessLaw P i)
  (fun n (r : Nat -> Fin k) => r n) measurability_proof r
```

4.3 The Directing Measure Uniqueness Lemma

A key innovation: if $\nu \leq C \cdot \mu$ for exchangeable probability measures, then their directing measures agree ν -almost everywhere.

Lemma 4.1 (`directingMeasure_singleton_ae_eq_of_smul_le`). Let μ, ν be probability measures with $X : \mathbb{N} \rightarrow \Omega \rightarrow \alpha$ exchangeable under both. If $\nu \leq C \cdot \mu$ for some $C < \infty$, then for all $b \in \alpha$:

$$\text{directingMeasure}_\mu(\omega)\{b\} =_{\nu\text{-a.e.}} \text{directingMeasure}_\nu(\omega)\{b\}$$

This is crucial for showing that the directing kernel computed from the full measure P serves correctly for restrictions $P|_{\{\omega_0=a\}}$.

4.4 Start-Restricted Factorization

The start-restricted invariance condition requires:

$$\text{rowProcessLaw}(P|_{\omega_0=a}, i) \text{ factors through } \text{directingRowKernel}(P, i)$$

The proof:

1. If $P(\omega_0 = a) = 0$: both sides are zero measures.
2. If $P(\omega_0 = a) = c > 0$: normalize $\rho_{a,\text{norm}} = c^{-1} \cdot \text{rowProcessLaw}(P|_{\omega_0=a}, i)$. This is a probability measure, and exchangeable (inherited).
3. Apply the i.i.d. de Finetti theorem to $\rho_{a,\text{norm}}$.
4. Apply Lemma 4.1: $\rho_{a,\text{norm}} \leq c^{-1} \cdot \text{rowProcessLaw}(P, i)$, so directing measures agree.

5. Scale back: the factorization identity holds for the unnormalized measure.

```

theorem startRestrictedRowKernelData_directingRowKernel
  (mu : PrefixMeasure (Fin k))
  (hmu : MarkovExchangeablePrefixMeasure mu)
  (P : Measure (Nat -> Fin k)) [IsProbabilityMeasure P]
  (hExt : forall xs, mu xs = P (cylinder xs))
  (hStrRec : StrongRecurrence P)
  (hExch : forall i, Exchangeable (rowProcessLaw P i)
    (fun n r => r n)) :
  StartRestrictedRowKernelData P (directingRowKernel P)

```

4.5 Carrier Transport

The start-restricted permutation invariance is derived from Markov exchangeability via finite combinatorics:

1. Fix state i , target $b \neq i$, and visit indices $n, n + 1$.
2. The *carrier* is the set of length- N prefixes consistent with a given visit-time event.
3. Construct a bijection between carriers by *segment swapping*: transpose the excursion between the n -th and $(n + 1)$ -th visits with the excursion between the $(n + 1)$ -th and $(n + 2)$ -th visits.
4. This bijection preserves Markov evidence (start state + transition counts).
5. By Markov exchangeability, the two carriers have equal μ -measure.
6. Taking $N \rightarrow \infty$ gives the start-restricted invariance.

```

theorem carrierTransportEquivAdjacent {N : Nat}
  (i : Fin k) (b : Fin k) (hbi : b != i) (n : Nat)
  (hSuff : sufficiency_hypotheses) :
  exists e : carrier(i, {n}, ..., N) <-> carrier(i, {n+1}, ..., N),
    forall xs, evidenceOf xs.1 = evidenceOf (e xs).1

```

5 Key Lemmas

5.1 Directing Measure Uniqueness

```

theorem directingMeasure_singleton_ae_eq_of_smul_le
  [Fintype alpha] [MeasurableSingletonClass alpha]
  (X : Nat -> Omega -> alpha) (hX_meas : forall n, Measurable (X n))
  {mu nu : Measure Omega} [IsProbabilityMeasure mu]
  [IsProbabilityMeasure nu]
  (hX_exch_mu : Exchangeable mu X)
  (hX_exch_nu : Exchangeable nu X)
  {C : ENNReal} (hC : C != top) (hle : nu <= C * mu)
  (b : alpha) :
  (fun omega => (directingMeasure (mu := mu) X hX_meas omega {b}).toReal)
  =ae[nu]
  (fun omega => (directingMeasure (mu := nu) X hX_meas omega {b}).toReal)

```

Proof idea: The directing measure is the a.e. limit of empirical frequencies. Under $\nu \leq C \cdot \mu$, any ν -a.e. property holds μ -a.e. on the support of ν . The empirical frequency limits agree on this common support.

5.2 Start-Restricted Row-Kernel Data

This is the theorem proved in the final session:

```

theorem startRestrictedRowKernelData_directingRowKernel
  (mu : PrefixMeasure (Fin k))
  (hmu : MarkovExchangeablePrefixMeasure mu)
  (P : Measure (Nat -> Fin k)) [IsProbabilityMeasure P]
  (hExt : forall xs, mu xs = P (cylinder xs))
  (hStrRec : StrongRecurrence P)
  (hExch : forall i, Exchangeable (rowProcessLaw P i)
    (fun n r => r n)) :
  StartRestrictedRowKernelData P (directingRowKernel P) := by
intro i a m sel hsel
by_cases hc : P {omega | omega 0 = a} = 0
-- Zero case: both sides are zero
case pos => ...
-- Positive case: normalize, apply de Finetti, use uniqueness
case neg =>
  set c := P {omega | omega 0 = a}
  set rho_a := rowProcessLaw (P.restrict {...}) i
  set rho_a_norm := c^(-1) * rho_a
  -- rho_a_norm is a probability measure
  -- rho_a_norm is exchangeable
  -- Apply de Finetti to rho_a_norm
  -- Apply directing measure uniqueness
  -- Scale back to rho_a
  ...

```

5.3 Carrier Transport Bijection

```

theorem carrierTransportEquivAdjacent {N : Nat}
  (i : Fin k) (b : Fin k) (hbi : b != i) (n : Nat)
  (hSuff : ...) :
  exists e : carrier(i, {n}, N) <-> carrier(i, {n+1}, N),
    forall xs, evidenceOf xs.1 = evidenceOf (e xs).1 := by
-- Construct segmentSwap
-- Prove self-inverse
-- Prove evidence preservation
-- Prove carrier membership preservation
...

```

6 File Inventory

All files are in Mettapedia/Logic/ within the Mettapedia Lean project.

6.1 External Dependencies

The formalization builds on the classical de Finetti theorem, formalized in Mettapedia/external/exchangeabi (43,683 lines, 0 sorry). This provides three independent proofs:

1. **Via reverse martingale convergence** (default, follows Kallenberg 2005)
2. **Via L^2 bounds** (lightest dependencies, elementary)
3. **Via mean ergodic theorem** (connects to dynamical systems)

Layer	File
Fiber bridge	MarkovDeFinettiFiberEventBridge.lean
Crux pipeline	MarkovDeFinettiFortiniBridgeCrux.lean
PE bridge	MarkovDeFinettiPEBridge.lean
Bridge core	MarkovDeFinettiFortiniBridgeCore.lean
Carrier transport	MarkovDeFinettiCarrierTransport.lean
Counterexample	MarkovDeFinettiFortiniCrossAnchorCounterexample.lean
Kernel uniqueness	MarkovDeFinettiKernelUniqueness.lean
Euler trails	MarkovDeFinettiHard*.lean (6 files)
Evidence basis	MarkovDeFinettiEvidenceBasis.lean
Moment problem	MarkovDeFinettiMomentProblem.lean
Higher-order Markov	MarkovDeFinettiHigherOrder.lean
Hidden Markov models	FiniteHiddenMarkovModel.lean
Higher-order Giry bridge	DeFinettiHigherOrderGiryBridge.lean
HMM de Finetti bridge	FiniteHiddenMarkovDeFinettiBridge.lean
Higher-order WM bridge	WMHigherOrderMarkov.lean
Controlled HMM/POMDP bridge	ControlledFiniteHiddenMarkovModel.lean, ControlledFiniteHiddenMarkovModel.lean
Controlled Solomonoff bridge	ControlledPrefixMeasure.lean, ControlledSolomonoffBridge.lean, ControlledSolomonoffBridge.lean
Other	(11 files)
Total	36 files

Table 1: Markov de Finetti formalization file inventory

7 The Classical de Finetti Context

The i.i.d. de Finetti theorem states:

$$\text{Contractable} \iff \text{Exchangeable} \iff \text{Conditionally i.i.d.}$$

for infinite sequences on standard Borel spaces.

The Markov version replaces “i.i.d.” with “time-homogeneous Markov chain” and “exchangeable” with “Markov-exchangeable.” The additional recurrence hypothesis is necessary to exclude deterministic transient behavior.

7.1 Row-Process Decomposition

The key bridge from Markov to i.i.d. is the *row-process*:

Definition 7.1. For anchor state i and trajectory ω , the *row-successor process* $\text{rsp}_i(\omega) : \mathbb{N} \rightarrow \text{Fin}(k)$ records the state visited *after* the n -th visit to i :

$$\text{rsp}_i(\omega, n) = \omega(t_{n,i}(\omega) + 1)$$

where $t_{n,i}(\omega)$ is the time of the n -th visit to state i .

Under successor-matrix PE, each row process $\text{rowProcessLaw}(P, i)$ is exchangeable in the classical sense. Applying the i.i.d. de Finetti theorem row-by-row extracts the directing measures that become the row kernels.

8 Toward Solomonoff: Structured Universal Mixtures

8.1 The Solomonoff Connection

Solomonoff’s universal prior assigns:

$$M(x_{1:n}) = \sum_{p:\text{program}} 2^{-|p|} \cdot \mathbf{1}[U(p) \text{ outputs } x_{1:n}]$$

This is a mixture over all computable hypotheses.

The Markov de Finetti theorem justifies a *structured* version:

$$\mu(x_{1:n}) = \int_{\Theta_k} \text{wordProb}_{\theta}(x_{1:n}) d\pi(\theta)$$

where π is a (possibly universal) prior on Markov parameters. Under Markov exchangeability, this representation exists and is unique.

8.2 Dominance Guarantees

A universal hyperprior over Dirichlet priors (one per transition row) gives:

- Conjugate Bayesian updates via count aggregation
- Dominance over all finitely parameterized Markov predictors
- Computable inference via the Dirichlet-multinomial formula

This is formalized in `MarkovDirichletHyperpriorMixture.lean`.

8.3 PLN Evidence Aggregation

The count-sufficiency result—under exchangeability, $\mu(xs)$ depends only on $\text{evidenceOf}(xs)$ —justifies PLN’s “evidence is what you carry” design. Binary evidence (n^+, n^-) is sufficient for exchangeable binary sequences; transition counts $N(i, j)$ are sufficient for Markov-exchangeable sequences.

8.3.1 The PLN-Markov Bridge

PLN (Probabilistic Logic Networks) represents uncertain knowledge via truth-value pairs (s, c) where s is strength and c is confidence. The Markov de Finetti theorem provides the measure-theoretic foundation for PLN’s evidence-based reasoning in sequential domains:

1. **Evidence sufficiency:** Under Markov exchangeability, the transition-count matrix $N(i, j)$ is a sufficient statistic. PLN’s evidence aggregation rules (count addition, forgetting via set difference) are justified by this sufficiency.
2. **Dirichlet conjugacy:** The posterior over Markov parameters given transition counts follows a product-Dirichlet distribution. This is formalized in `EvidenceDirichlet.lean` and underlies PLN’s Bayesian update rules.
3. **Predictive inference:** Given evidence N , the predictive distribution for the next transition $i \rightarrow j$ is $\frac{N(i, j) + \alpha}{\sum_k N(i, k) + k\alpha}$ (Laplace smoothing). This is the “strength” in PLN terms, with confidence derived from total count.

The formalization connects these three layers: the representation theorem (§4) justifies evidence sufficiency, the Dirichlet infrastructure (`MarkovDirichletPredictor.lean`) provides conjugate updates, and the hyperprior mixture (`MarkovDirichletHyperpriorMixture.lean`) gives Solomonoff-style dominance guarantees.

Current theorem surface (2026-04): the post-Markov bridge is now exposed at three abstraction levels:

1. **General k -ary:** `markovExchangeableDomainCharacterization` shows the sufficient statistic is `(counts, last)` (`MarkovExchangeabilityBridge.lean`).
2. **Binary presentations:** Both `Fin 2` and explicit Boolean `(ff, ft, tf, tt, lastBit)` with witness-based Solomonoff wrappers and direct Markov ν PLN master-chain theorems.
3. **Class-recurrence:** `MarkovBinaryPLNDomainConditionsInClass` packages binary domain conditions under class recurrence, with `Fin 2` and `Bool` row-law factorization theorems (`SolomonoffExchangeable.lean:590--802`). The downstream theorems are scoped to $i \in C$, which is the honest mathematical scope.

The class-recurrence layer enables PLN-style reasoning in settings where only partial recurrence is known, with explicit tracking of which transition rows support the factorization.

8.3.2 The Markov Xi Transition Surface

The formalization exposes a direct transition atom surface via `MarkovTransitionXi.lean` and `MarkovPathXi.lean`. Key theorem: single-step transition queries compile to row-conditioned Dirichlet posterior means (`markovTransitionAtom.semEq_rowProjection`).

Multi-step chaining. PLN’s standard deduction screening (`CarrierDeductionScreeningOff`) assumes evidence additivity:

$$\text{derive}(W) = \text{proj}(\text{view}(A \rightarrow B) + \text{view}(B \rightarrow C))$$

For Markov chains, this is *unfillable*. The screening condition $\text{proj}(\text{rowA} + \text{rowB}) = \text{proj}(\text{rowA})$ holds only when $\text{rowB} = 0$.

The honest alternative is **predictive sequential composition** (`MarkovPredictiveChaining.lean`):

$$P(x_n | x_0, \text{counts}) = \sum_{x_1, \dots, x_{n-1}} \prod_{i=0}^{n-1} P(x_{i+1} | x_i, \text{counts})$$

This is formalized as `markovWMPosteriorChain` and exposed via the `markov-transition*` atom family (`MarkovPathXi.lean:43`).

Concrete examples. The binary path atoms `bit001` ($0 \rightarrow 0 \rightarrow 1$) and `bit011` ($0 \rightarrow 1 \rightarrow 1$) demonstrate the full pipeline from transition multiset to predictive chain probability (`MarkovPathXi.lean:290--327`).

9 The Structural Audit: Bug-Hunting Methodology

A structural audit in April 2026 revealed three classes of bugs in the proof architecture that had accumulated during development. This section documents the methodology and findings, as a reference for future large-scale formalizations.

9.1 The Problem: Assumption Interfaces Mask Gaps

The formalization initially used `defs` to encode partial results:

```
-- ANTI-PATTERN: defined as Prop, never instantiated
def BuildRowKernelOnRecurrentExtension (k : Nat) : Prop :=
  forall mu P, MarkovExchangeablePrefixMeasure mu -> ...
    exists rowKernel, BuiltRowKernelOnExtension P rowKernel

def SuccessorMatrixPE_of_markovExchangeable_recurrent
  (k : Nat) : Prop := ...
```

These `defs` compiled with zero `sorry`, but they were *never instantiated*. The actual mathematical content was encoded as assumptions, not proved theorems. A grep for `sorry` showed zero hits, creating a false sense of completeness.

9.2 Bug 1: Evidence-Preserving Equiv Used Below Full Strength

The carrier transport bijection preserves the *complete* Markov evidence (start state + all k^2 transition counts). But the pipeline extracted only *1D marginal invariance*:

```
-- What was extracted:
P({omega0=a} inter {succ at visit sigma(n) to i = b})
  = P({omega0=a} inter {succ at visit n to i = b})

-- What the Equiv actually proves:
forall xs in carrier, evidenceOf xs = evidenceOf (e xs)
```

The 1D extraction suffices for some arguments but discards the joint finite-dimensional invariance needed for `SuccessorMatrixPartialExchangeable`.

Fix: Upgrade the pipeline to extract the full evidence-preservation property, then derive `SuccessorMatrixPE` via the π - λ theorem (finite cylinder sets generate the product σ -algebra).

9.3 Bug 2: Zero-Sorry Masking of Unproved Content

The three `defs` listed above (`BuildRowKernelOnRecurrentExtension`, `SuccessorMatrixPE_of_markovExchangeable`, `ExistsBuiltRowKernel_of_successorMatrixPE`) bundled the genuine mathematical content into their *types*, not their proofs. They were defined as `Props` and used as hypotheses in downstream theorems, creating a sorry-free but mathematically incomplete development.

Fix: Replace `defs` with explicit `theorem` statements that must be proved. The type system then forces instantiation.

9.4 Bug 3: The Euler Trail Archive Was the Missing Bridge

The `_archive/MarkovDeFinetti/` directory contained 3,265 lines of fully proved infrastructure (0 sorry) for trajectory decomposition via Euler trails:

- `CopyPerm`: Edge-copy permutations preserve vertex sequences
- `EulerTrails`: Trail count = $\prod_{a,b} G(a,b)!$
- `trailVertexSeq_applyCopyPerm`: Copy perms preserve trajectories

This infrastructure provides *global* carrier bijections (not just adjacent transpositions), directly giving `SuccessorMatrixPartialExchangeable`. But it was archived and not imported into the active development.

Fix: Port the `segmentSwap` construction to the active framework. The key theorem `carrierTransportEquiv` (1,374 lines) provides the adjacent-visit bijection; composition via `Equiv.trans` and `Nat` induction extends to arbitrary visit indices.

9.5 Audit Methodology

1. **Trace the theorem DAG:** Starting from the crown theorem, recursively list all dependencies. Mark each as “proved” (has a proof term) or “assumed” (is a hypothesis or def).
2. **Grep for assumption interfaces:** Search for defs of type `Prop` that are never instantiated. These are masked gaps.
3. **Check archive for reusable infrastructure:** Archived code often contains fully proved lemmas that were set aside during refactoring. A grep for the required property names may find them.
4. **Verify the full pipeline:** Build every file independently (`lake env lean -j1 <file>`) to catch import-order issues.

10 Completed Extensions

Beyond the core Markov de Finetti theorem, we formalized several extensions that complete the story for practical applications.

10.1 Abstract Recurrence Generalization

The core theorem requires recurrence to the *starting state*. We generalized this to arbitrary *recurrent classes*—subsets of the state space where every pair of states communicates infinitely often:

```
def RecurrentClass (P : Measure (Nat -> Fin k)) (C : Set (Fin k)) : Prop :=
  C.Nonempty /\
  forall i j : Fin k, i in C -> j in C ->
    ae omega : P, Set.Infinite {t | omega t = i /\ omega (t+1) = j}

def StrongRecurrenceInClass (C : Set (Fin k)) (P : Measure ...) : Prop :=
  forall i in C, ae omega : P,
    (exists t, omega t in C) -> Set.Infinite {t | omega t = i}
```

The key insight is that class recurrence provides guarantees only for rows *within* the class. We built the full theorem infrastructure for this setting:

- `nthVisitTimeExists_of_StrongRecurrenceInClass` bridges class-level recurrence to the row-process machinery
- `ae_rowRecurrence_of_StrongRecurrenceInClass` extracts the exact row-specific recurrence predicate needed by the PE bridge
- `exchangeable_rowProcess_of_rowRecurrence` proves row-process exchangeability from the row-specific hypothesis

The public API packages these results in `ClassRestrictedMarkovMixtureSurface`, which carries the extension law, class recurrence hypothesis, and all derived row-law consequences. The theorem `ClassRestrictedMarkovMixtureSurface.class_transition_structure` provides the canonical entrypoint for downstream applications.

10.2 The MarkovMixture API

For downstream users, we provide a clean interface packaging the theorem:

```
structure MarkovMixture (k : Nat) (mu : PrefixMeasure (Fin k)) where
  mixingLaw : Measure (MarkovParam k)
  mixingLaw_prob : IsProbabilityMeasure mixingLaw
  represents : forall xs, mu xs = integral theta, wordProb theta xs dmixingLaw
```

Multiple constructors build `MarkovMixture` objects from different theorem surfaces: `of_extension_strongR` uses the direct extension route, while `of_successorMatrixPE_minimal` uses the minimal PE payload. For class-restricted settings, `ClassRestrictedMarkovMixtureSurface` provides the analogous object with honest scope for rows in the class.

10.3 Kleisli(Giry) Prefix-Law Bridge

We also connected the measure-theoretic theorem surface to the existing categorical Giry infrastructure. The new file `CategoryTheory/DeFinettiMarkovGiryBridge.lean` packages a `MarkovMixture` as a constant `Kleisli(Giry)` mediator

$$1 \rightarrow \mathcal{G}(\text{MarkovParam}_k),$$

and proves that the finite-prefix law factors through this mediator. The key theorems are:

- `kleisliGiryMarkovDeFinettiFactorization_of_markovMixture`
- `categoricalMarkovDeFinettiFactorization_iff_kleisliGiry`
- `kleisliGiryMarkovDeFinettiFactorization_iff_nonempty_markovMixture`

This gives an honest categorical restatement of the proved theorem surface: the public Markov-mixture witness is equivalent to a `Kleisli(Giry)` factorization through the Markov parameter space at the finite-prefix level.

10.4 Binary Post-Markov Bridge

For applications to PLN and Solomonoff-style prediction, we formalized the binary specialization at two presentation levels:

1. **Fin 2 presentation:** Transition counts as a 2×2 matrix `TransCounts 2`, with theorems like `markovExchangeable_domain_characterization_fin2`
2. **Bool presentation:** Explicit counts (ff, ft, tf, tt) plus last bit, with the equivalence theorem `binarySummary_eq_iff_fin2Summary_eq` bridging the two

The class-recurrence variant `MarkovBinaryPLNDomainConditionsInClass` packages the binary domain conditions under class recurrence, enabling PLN-style reasoning when only partial recurrence is known. The Solomonoff bridge uses `RestrictedSolomonoffMarkovWitness`—an honest witness object rather than an overstated universal claim—with direct ν PLN master-chain theorems packaging predictor sufficiency, shared Markov-Dirichlet update states, and standard regret bounds.

10.5 Borel-Measurable Sequence Kernel

The prefix-law Giry bridge (§9.3) factors `MarkovMixture` through a Kleisli mediator at the *finite-prefix* level. For applications requiring infinite sequence measures, we constructed the stronger object: a measurable kernel

$$\text{MarkovParam}_k \rightarrow \mathcal{G}((\text{Fin } k)^{\mathbb{N}})$$

on the *Borel* parameter space.

The construction uses a paired-trajectory approach. Rather than directly sampling states, we track pairs $(\theta, s_n) \in \text{MarkovParam}_k \times \text{Fin } k$ through time, where θ is carried deterministically and s_n evolves according to $\theta.\text{trans}$:

```
-- Paired transition kernel
def markovPairStepKernel : Kernel (PairState k) (PairState k) :=
  (keepParamKernel || markovStepKernel) . Kernel.copy

-- Full trajectory via Mathlib's traj API
def markovPairedTrajectoryKernel : Kernel (MarkovParam k) (PairTrajectory k) :=
  (Kernel.traj pairPrefixKernel 0) . pairStartPrefixKernel

-- Project to state sequences
def markovSequenceKernel : Kernel (MarkovParam k) (Nat -> Fin k) :=
  Kernel.map markovPairedTrajectoryKernel stateSequenceOfPairTrajectory
```

The critical measurability lemmas are established in `Logic/MarkovDeFinettiMomentProblem.lean`:

- `continuous_initProb_borel`: $\theta \mapsto \theta.\text{init}(a)$ is continuous
- `continuous_stepProb_borel`: $\theta \mapsto \theta.\text{trans}(a, b)$ is continuous
- `measurable_initProb_borel`, `measurable_stepProb_borel`: Borel measurability follows

The new file `Logic/MarkovDeFinettiSequenceKernelBorel.lean` (1055 lines) builds the full kernel with `IsMarkovKernel` instances at each stage, including the critical theorem:

```
theorem markovSequenceKernel_cylinder_eq_wordProb (theta : MarkovParam k) :
  forall xs, markovSequenceKernel theta (cylinder xs) = wordProb theta xs
```

10.6 Categorical Borel-Side Surface

The Borel sequence kernel enables a stronger categorical factorization. Given a probability measure π on MarkovParam_k (with Borel σ -algebra), we define the *Borel sequence mixture*:

$$\mu_\pi := \int_{\text{MarkovParam}_k} \text{markovSequenceKernel}(\theta) d\pi(\theta)$$

This is implemented as:

```
def borelMarkovSequenceMixture (pi : ProbMarkov k) : Measure (Nat -> Fin k) :=
  pi.bind markovSequenceKernel

theorem borelMarkovSequenceMixture_cylinder_eq (pi : ProbMarkov k) (xs : List (Fin k))
  :
  borelMarkovSequenceMixture pi (cylinder xs) =
    integral theta, markovSequenceKernel theta (cylinder xs) dpi
```

The public factorization surface `CategoricalBorelMarkovSequenceKernelFactorization` packages this as a categorical statement: the prefix law μ factors through integrating the sequence kernel against a Borel probability law on parameters.

One-Way Lift: The Borel-side construction uses a finer σ -algebra than the active `markovParamMeasurable` (generated by `wordProb` coordinates). We prove the key bridge theorem connecting the two surfaces:

```
-- Trim a Borel law to the active sigma-algebra
def probMarkovToActiveMeasure (pi : ProbMarkov k) : Measure (MarkovParam k) :=
  pi.trim wordProbGenerated_le_borel

-- Trimming preserves wordProb integrals
theorem lintegral_probMarkovToActiveMeasure_wordProb (pi : ProbMarkov k) (xs : List (Fin k)) :
  integral theta, wordProb theta xs d(probMarkovToActiveMeasure pi) =
    integral theta, wordProb theta xs dpi

-- One-way lift: BorelMarkovMixture -> MarkovMixture
def markovMixtureOfBorelMarkovMixture (M : BorelMarkovMixture k mu) : MarkovMixture k mu
```

This is a *one-way* lift: every `BorelMarkovMixture` yields an active `MarkovMixture` by trimming the mixing law.

10.7 Non-Uniqueness of Borel Lifts

The converse direction—lifting an active `MarkovMixture` to a Borel representative—is *not uniquely determined*. We prove this via an explicit “dead row” counterexample in the binary case:

```
-- Two distinct binary parameters with identical finite-word laws
def deadRowTheta_0 : MarkovParam 2 where -- row 1 -> 0
  init := dirac 0
  trans | 0 => dirac 0 | 1 => dirac 0

def deadRowTheta_1 : MarkovParam 2 where -- row 1 -> 1
  init := dirac 0
  trans | 0 => dirac 0 | 1 => dirac 1

theorem deadRowTheta_0_ne_deadRowTheta_1 : deadRowTheta_0 /= deadRowTheta_1
theorem deadRowTheta_wordProb_eq (xs : List (Fin 2)) :
  wordProb deadRowTheta_0 xs = wordProb deadRowTheta_1 xs
```

Since state 0 is absorbing and we always start there, row 1 is never reached. The two parameters differ only on the unreachable row, so their finite-word observables are identical. This lifts to:

```
theorem not_injective_borelMomentMapWord_fin2 :
  not Function.Injective (fun pi => fun xs => momentMapWord xs pi)

theorem deadRowPrefixMeasure_has_two_distinct_borelMarkovMixtures :
  deadRowBorelMarkovMixture_0 /= deadRowBorelMarkovMixture_1
```

Consequence: Any converse from active prefix data to Borel witnesses cannot be canonical/unique without additional hypotheses (e.g., recurrence that makes all rows reachable). Whether a mere *existence* converse holds by measure extension remains open.

10.8 Higher-Order Markov Chains

An order- m Markov chain on alphabet `Fin k` can be reduced to an ordinary first-order chain on *context states*—length- m windows of symbols. This reuses the verified finite-state machinery directly:

```

-- Context states of length m
abbrev Context (k m : Nat) := Fin m -> Fin k

-- Higher-order parameter: initial law on contexts + next-symbol kernel
structure HigherOrderMarkovParam (k m : Nat) where
  init : ProbabilityMeasure (Context k m)
  next : Context k m -> ProbabilityMeasure (Fin k)

-- Reduction to ordinary MarkovParam on encoded contexts
noncomputable def toMarkovParam (theta : HigherOrderMarkovParam k m) :
  MarkovParam (Fintype.card (Context k m))

```

The key theorem `contextSequenceMeasure_cylinder_eq_contextWordProb` proves that cylinder events on context trajectories evaluate to the reduced `wordProb`—a direct port of the first-order sequence theorem through the context-state encoding.

For evidence, we define $(m + 1)$ -gram counts:

```

structure GramCounts (k m : Nat) where
  counts : Context k m -> Fin k -> Nat

def gramCountsOfPath : List (Context k m) -> GramCounts k m
def summary (xs : List (Fin k)) : Option (GramCounts k m * Context k m)

```

The `summary` function extracts $(m + 1)$ -gram counts plus the final context once the observed word is at least m symbols long. Short prefixes are handled by the initial context law directly.

The key incremental update theorem `summary_append_singleton` proves that appending a symbol updates the summary by bumping the appropriate $(m + 1)$ -gram count and shifting the terminal context.

For the raw-symbol sequence theorem, we establish shift-consistency predicates and a zero-mass support lemma:

```

def consistentTail (ctx : Context k m) : List (Context k m) -> Prop
def prefixShiftConsistent : List (Context k m) -> Prop

theorem contextWordProb_eq_zero_of_not_prefixShiftConsistent
  (theta : HigherOrderMarkovParam k m)
  (ctxs : List (Context k m))
  (hbad : not (prefixShiftConsistent ctxs)) :
  contextWordProb theta ctxs = 0

```

The raw-symbol sequence theorem is now fully proved:

```

theorem higherOrderSequenceMeasure_cylinder_eq_longWordProb
  (theta : HigherOrderMarkovParam k m) (xs : List (Fin k)) (hxs : m <= xs.length) :
  higherOrderSequenceMeasure theta (cylinder xs) = longWordProb theta xs hxs

```

Concrete order-2 regression examples demonstrate the full pipeline:

```

theorem orderTwo_cylinder_001_eq_longWordProb (theta : HigherOrderMarkovParam 2 2) :
  higherOrderSequenceMeasure theta (cylinder [0, 0, 1]) = longWordProb theta [0, 0, 1]

theorem orderTwo_cylinder_011_eq_longWordProb (theta : HigherOrderMarkovParam 2 2) :
  higherOrderSequenceMeasure theta (cylinder [0, 1, 1]) = longWordProb theta [0, 1, 1]

```

- **Positive example:** the cylinder $[0, 0, 1]$ on an order-2 binary chain now reduces to the honest higher-order probability surface via the full raw-symbol theorem.

- **Negative example:** this does not give de Finetti mixtures for higher-order chains; that requires lifting through the Borel/Giry bridge.

File: MarkovDeFinettiHigherOrder.lean (1,219 lines, 0 sorry).

10.9 Finite Hidden Markov Models

A finite HMM is a latent finite Markov chain plus a finite emission kernel:

```
structure FiniteHMMParam (latent obs : Nat) where
  latentParam : MarkovParam latent
  emission : Fin latent -> ProbabilityMeasure (Fin obs)

def emissionProb (theta : FiniteHMMParam latent obs) (x : Fin latent) (y : Fin obs) :
  NNReal :=
  theta.emission x (Set.singleton y)
```

The finite-word observation probability sums over all latent words:

```
def observedWordProb (theta : FiniteHMMParam latent obs) (ys : List (Fin obs)) :
  ENNReal :=
  sum xs : Fin ys.length -> Fin latent,
    wordProb theta.latentParam (List.ofFn xs) *
    observationWeight theta (List.ofFn xs) ys
```

Beyond finite-word sums, the formalization now provides a genuine observed sequence measure via a paired latent/observation construction:

```
-- Paired latent/observation state
abbrev PairState (latent obs : Nat) := Fin latent * Fin obs

-- Paired-state Markov parameter
noncomputable def pairedMarkovParam (theta : FiniteHMMParam latent obs) :
  MarkovParam (Fintype.card (PairState latent obs))

-- Genuine observed sequence measure (pushforward to observations)
noncomputable def observedSequenceMeasure (theta : FiniteHMMParam latent obs) :
  Measure (Nat -> Fin obs) :=
  (pairedSequenceMeasure theta).map observationSequenceOfPairedTrajectory

instance observedSequenceMeasure_isProbability :
  IsProbabilityMeasure (observedSequenceMeasure theta)
```

The observation-cylinder theorem connects the sequence measure back to finite-word probabilities:

```
theorem observedSequenceMeasure_cylinder_eq_observedWordProb
  (theta : FiniteHMMParam latent obs) (ys : List (Fin obs)) :
  observedSequenceMeasure theta (cylinder ys) = observedWordProb theta ys
```

Key results:

- `observedWordProb_nil`: empty observation has probability 1
- `emissionKernel`: the emission kernel as an explicit Kernel
- `observedSequenceMeasure`: genuine sequence law on observations
- `observedSequenceMeasure_cylinder_eq_observedWordProb`: cylinder theorem
- `pairedWordProb_eq`: paired word factorization into latent and emission

This gives a complete finite HMM sequence-law surface: the sequence measure evaluates cylinder events exactly as the finite-word observation probability.

- **Positive example:** a 3-state latent chain with 5-symbol emissions has `observedSequenceMeasure (cylinder [y0,y1,y2]) = observedWordProb [y0,y1,y2]`.
- **Negative example:** this does not give Baum-Welch, forward-backward, or posterior inference; what we proved is the finite sequence-law surface.

File: `FiniteHiddenMarkovModel.lean` (668 lines, 0 sorry).

10.10 Higher-Order Borel/Giry Bridge

The higher-order raw-cylinder theorem lifts into the existing Borel/Giry layer by reducing order- m chains to ordinary first-order chains on finite encoded context states:

```
def higherOrderLongWordWeightViaProbMarkov
  (pi : ProbMarkov (HigherOrderContextCard k m))
  (xs : List (Fin k)) (hxs : m <= xs.length) : ENNReal :=
  borelMarkovCylinderWeightViaProbMarkov pi (higherOrderEncodedContextWord xs hxs)

def CategoricalBorelHigherOrderLongWordFactorization (k m : Nat) (mu : PrefixMeasure)
  : Prop :=
  exists pi : ProbMarkov (HigherOrderContextCard k m),
  forall xs, forall hxs : m <= xs.length,
    mu xs = higherOrderLongWordWeightViaProbMarkov pi xs hxs
```

The design keeps the boundary explicit: the mediator is an ordinary Borel `MarkovParam` on encoded contexts, not a new measurable space on `HigherOrderMarkovParam` itself.

File: `DeFinettiHigherOrderGiryBridge.lean` (175 lines, 0 sorry).

10.11 Finite HMM de Finetti Bridge

The finite HMM surface also lifts to the Borel layer, with the latent parameter law as a Borel probability measure on `MarkovParam` and the emission kernel held fixed:

```
theorem measurable_observedWordProb_borel
  (emission : Fin latent -> ProbabilityMeasure (Fin obs))
  (ys : List (Fin obs)) :
  Measurable (fun theta => observedWordProb { latentParam := theta, emission } ys)

def CategoricalBorelFiniteHMMFactorization (latent obs : Nat)
  (emission : Fin latent -> ProbabilityMeasure (Fin obs))
  (mu : PrefixMeasure (Fin latent)) : Prop :=
  exists pi : ProbMarkov latent,
  forall ys, mu ys = observedWordWeightViaProbMarkov emission pi ys
```

Includes concrete binary regression examples (`binaryCopyHMM_observedCylinder_0/1`).

File: `FiniteHiddenMarkovDeFinettiBridge.lean` (339 lines, 0 sorry).

10.12 Controlled Finite HMMs, Credal Beliefs, and POMDP-Style Planning

The action-conditioned extension turns the finite HMM surface into a structured finite POMDP substrate. The latent transition kernel now depends on an externally supplied action:

```
structure ControlledFiniteHMMParam (Action : Type) (latent obs : Nat) where
  init : ProbabilityMeasure (Fin latent)
  trans : Action -> Fin latent -> ProbabilityMeasure (Fin latent)
  emission : Fin latent -> ProbabilityMeasure (Fin obs)
```

At the observation level, the formalization proves a complete finite-state filtering and smoothing surface for completed action-observation cycles:

- `filteringMass`: unnormalized latent belief after a completed trace,
- `filteringPosteriorMass`: normalized posterior given a nonzero observation prefix,
- `smoothingPosteriorMass`: two-sided latent posterior for a split completed trace,
- `no_observedOnly_multiset_queryStrength_recovers_filteringPosteriorMass`: an honest obstruction theorem showing that unordered observed multisets do not determine hidden-state beliefs.

This is the key boundary for WM/PLN reasoning under partial observability: observation-only additive evidence is insufficient for latent posterior tracking, so the formalization moves to a *credal* world-model layer. For a finite family of controlled HMMs, the interval-valued object `filteringCredalPLNITV` packages lower and upper latent-state strengths as a live indefinite truth value. Recursive lower and upper optimal-value envelopes then propagate that ambiguity into decision theory.

The planning bridge itself is concrete. A controlled finite HMM is turned into a `BayesianAgents.Environment` and the resulting value and `qValue` theorems exercise the existing AIXI-style finite-horizon planning layer. The capstone ambiguity theorem shows:

1. each family member's finite-horizon `optimalValue` lies inside the recursive credal interval,
 2. the interval can be nontrivially wide, and
 3. there need not exist a single observation-history-based optimal action shared by all family members.
- **Positive example**: the ambiguous controlled family widens the horizon-4 optimal-value interval while preserving identical observation behaviour.
 - **Negative example**: this does not yet identify a unique hidden state or unique optimal action from observations alone.

Files: `ControlledFiniteHiddenMarkovModel.lean`, `ControlledFiniteHiddenMarkovObservedInference.lean`, `WMControlledFiniteHiddenMarkovCredal.lean`, `ControlledFiniteHiddenMarkovBridge.lean`, `ControlledFiniteHiddenMarkovPlanningExamples.lean`, `ControlledFiniteHiddenMarkovExports.lean`.

10.13 Controlled Solomonoff Regret Along Fixed Action Streams

The new universal-prediction leg for the controlled setting is conservative but honest. Rather than claiming a fully action-aware universal prior, the formalization lifts the ordinary finite-alphabet Solomonoff semimeasure on the observation alphabet into the controlled setting by *ignoring the action side-information*:

```
noncomputable abbrev controlledM2 : ControlledSemimeasure Action Y :=
  ControlledSemimeasure.ofObservationSemimeasure
    (SolomonoffBridge.M2_on_observations)
```

Conditioning this controlled semimeasure on any fixed action stream recovers the ordinary observation-alphabet universal mixture M_2 . Consequently, for any controlled prefix law whose conditioned observation process is lower-semicomputable, the standard finite-horizon Solomonoff log-loss regret bound applies along that fixed action stream.

For controlled finite HMMs, this yields a concrete theorem surface, schematically:

```

theorem relEntropy_le_log_inv_controlledM2
  (theta : ControlledFiniteHMMPParam Action latent obs)
  (u : Nat -> Action)
  (hmu : LowerSemicomputablePrefixMeasure
    ((toControlledPrefixMeasure theta).conditionOnActionStream u))
  (n : Nat) :
  exists c, c != 0 /\
    Dominates (controlledM2.conditionOnActionStream u)
      ((toControlledPrefixMeasure theta).conditionOnActionStream u) c /\
    ControlledFiniteHorizon.relEntropy
      (toControlledPrefixMeasure theta) controlledM2 u n
      <= log (1 / c.toReal)

```

The lower-semicomputability hypothesis is not an implementation leftover. The current parameter record allows arbitrary probability measures, so computability of the induced law is a genuine extra domain condition. Removing that hypothesis would require an additional computable-parameter surface, not a proof cleanup.

10.14 Higher-Order WM/PLN Bridge

The $(m + 1)$ -gram summary threads into the WM/PLN additive evidence layer:

```

abbrev HigherOrderObservation (k m : Nat) := Context k m * Fin k

def rowEvidence (c : GramCounts k m) (ctx : Context k m) : MultiEvidence k :=
  { counts := fun a => c.counts ctx a }

def higherOrderRowStatistic :
  SufficientStatisticSurface (HigherOrderObservation k m) (Context k m) (
    MultiEvidence k)

```

The design stays honest: additive evidence is context-conditioned next-symbol counts; the final context is boundary data not forced into the additive carrier.

File: `WMHigherOrderMarkov.lean` (365 lines, 0 sorry).

11 Conclusion

We have presented a complete Lean 4 formalization of the Markov de Finetti theorem (Diaconis–Freedman 1980), comprising 26,665 lines with zero unproven assumptions. Key innovations include:

- A directing measure uniqueness lemma that transfers de Finetti structure across measure restrictions.
- A carrier transport bijection via trajectory segment swapping.
- A structural audit methodology that identified and closed previously overlooked gaps in the proof architecture.

The formalization provides a verified foundation for structured universal mixtures, connecting Solomonoff-style prediction to tractable Markov models with formal correctness guarantees.

11.1 Lessons Learned

1. **Gap analysis pays off:** A structural audit in April 2026 identified three bugs in the proof architecture that had been masked by assumption interfaces. Explicit theorem statements (not defs) prevent this.

2. **Reuse existing formalizations:** The 43,683-line de Finetti infrastructure was essential. Building row kernels from the directing measure machinery avoided reimplementing measure-theoretic foundations.
3. **Uniqueness lemmas are load-bearing:** The directing measure uniqueness lemma (§5.1) is small (100 lines) but enables the entire start-restricted factorization argument.

11.2 Future Directions

Two directions remain open for future work:

11.2.1 Existence of Non-Unique Borel Lift

The Giry integration (§9.5–9.7) is now complete except for one question: does every active `MarkovMixture` have *some* Borel lift, even though it is not unique? The non-uniqueness counterexample (§9.7) shows that uniqueness fails, but existence by measure extension on compact Polish spaces may still hold. This would require Mathlib infrastructure for extending measures from sub- σ -algebras on compact metrizable spaces.

11.2.2 Computational Extraction

The proof constructs the mixing measure as a pushforward; this is non-constructive (uses classical choice for directing measures). A computational variant would:

- Compute finite-horizon approximations to the mixing measure
- Provide error bounds in total variation or Wasserstein distance
- Connect to MCMC sampling of the posterior over Markov parameters

11.2.3 Extended State Spaces

- **Countable alphabets:** Extend to $\mathbb{N}$ -valued sequences. The main obstacle is the lack of a uniform probability measure on $\mathbb{N}$; the proof would need to work with sub-probability kernels.
- **Continuous state spaces:** The standard Borel case requires disintegration theory (regular conditional probabilities). Mathlib’s `MeasureTheory.Measure.condexp` provides conditional expectation but not full disintegration.
- **Controlled hidden Markov models:** the action-conditioned finite POMDP substrate, credal decision envelopes, and fixed-action-stream Solomonoff regret bounds are now formalized; what remains is the genuinely action-aware universal mixture and a controlled de Finetti theorem.
- **Higher-order Markov:** Extend to k -th order Markov chains where the sufficient statistic is $(k + 1)$ -gram counts.

Acknowledgments

This formalization builds on Mathlib and the exchangeability library. We thank the Lean and Mathlib communities for the infrastructure that makes such formalizations possible.

References

- [1] P. Diaconis and D. Freedman. De Finetti's theorem for Markov chains. *Ann. Probab.*, 8(1):115–130, 1980.
- [2] B. de Finetti. La prévision: ses lois logiques, ses sources subjectives. *Ann. Inst. Henri Poincaré*, 7:1–68, 1937.
- [3] O. Kallenberg. *Probabilistic Symmetries and Invariance Principles*. Springer, 2005.
- [4] R. J. Solomonoff. A formal theory of inductive inference. *Inform. Control*, 7(1–2):1–22, 224–254, 1964.
- [5] The mathlib Community. The Lean mathematical library. <https://github.com/leanprover-community/mathlib4>, 2024.